UNIVERSITY OF NEBRASKA OMAHA

Computer Science

CSCI8590-001: Fundamentals Of Deep Learning

Instructor: Dr. *Xin Zhong*

Assignment 3

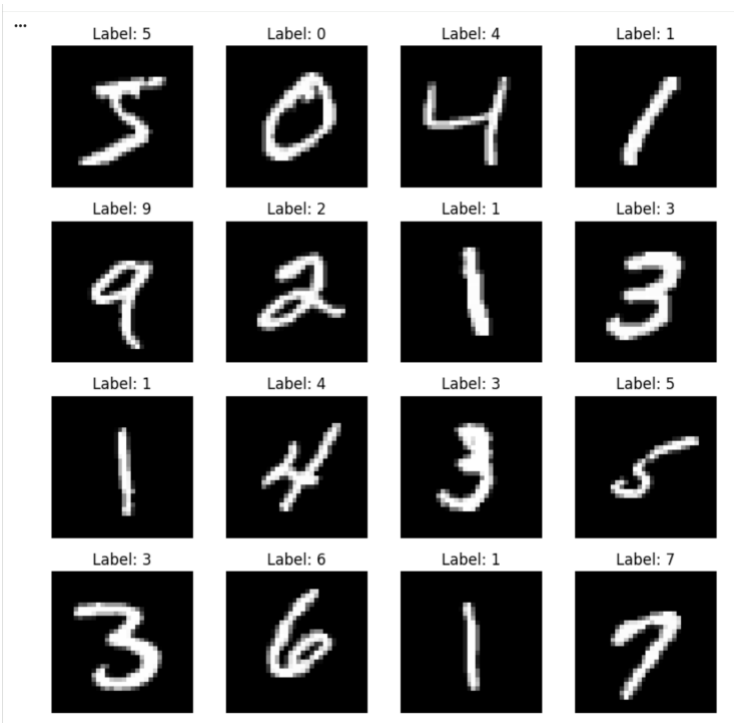
Submitted By: Zerin Shaima Meem

NUID: 77548102

Runnable Code: Google Colab Link

Required Output:

Sample images:



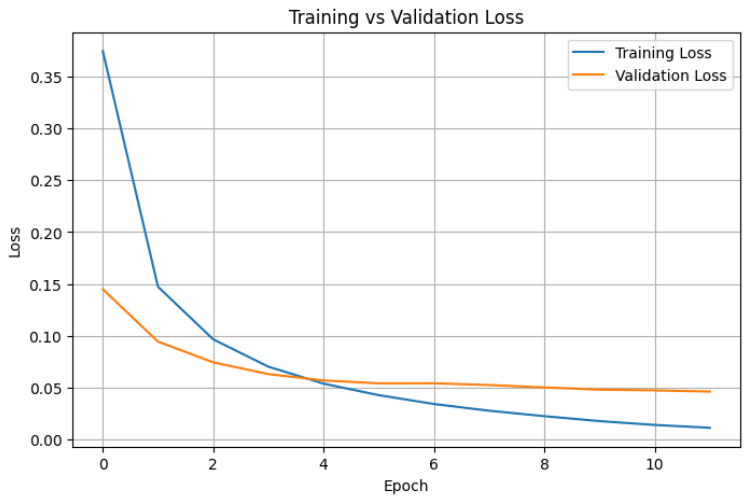
Model summary:

Model: "LeNet5"

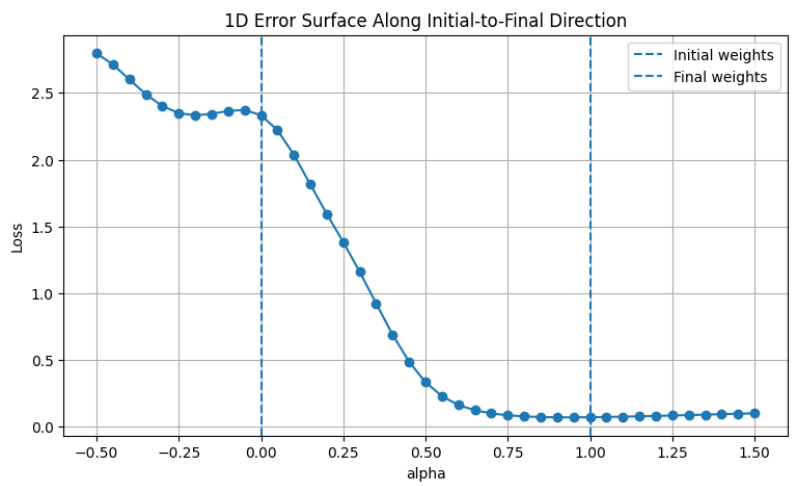
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 32, 32, 1)	0
conv2d (Conv2D)	(None, 28, 28, 6)	156
average_pooling2d (AveragePooling2D)	(None, 14, 14, 6)	0
conv2d_1 (Conv2D)	(None, 10, 10, 16)	2,416
average_pooling2d_1 (AveragePooling2D)	(None, 5, 5, 16)	0
conv2d_2 (Conv2D)	(None, 1, 1, 120)	48,120
flatten (Flatten)	(None, 120)	0
dense (Dense)	(None, 84)	10,164
dense_1 (Dense)	(None, 10)	850

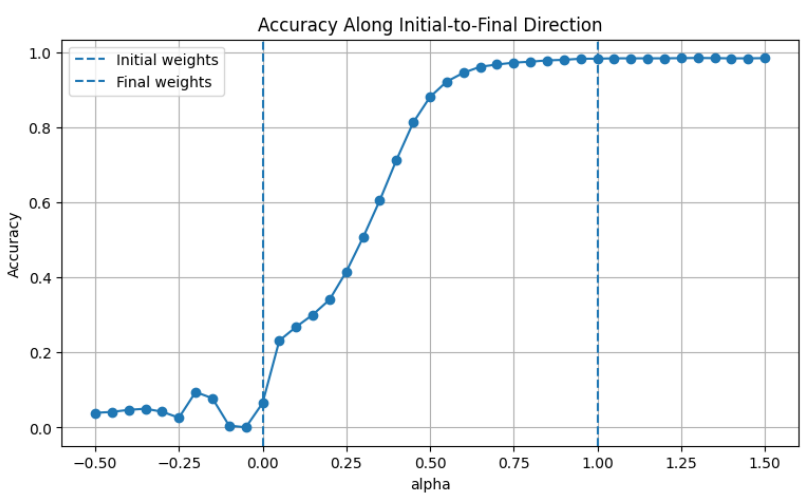
Total params: 61,706 (241.04 KB)
Trainable params: 61,706 (241.04 KB)
Non-trainable params: 0 (0.00 B)

Training curves:



Error surface plot(s):





Discussion:

Interpretation of result: The first output figure titled "1D Error Surface Along Initial-to-Final Direction" shows how the **loss changes along the direction from the initial weights to the trained weights**. At $\alpha = 0$ (initial weights), the loss is very high (around 2.3), which is expected because the network starts with random parameters and performs close to random guessing. As **α** increases, the loss decreases smoothly, indicating that moving toward the trained parameters steadily improves the model. The minimum loss appears around α between 0.8 and 1.0, where the curve becomes relatively flat. This shows that the model converged to a **wide and stable minimum**, meaning small changes in the parameters near the final solution do not significantly increase the loss.

The second figure of "Accuracy Along Initial-to-Final Direction", shows the **accuracy along the same interpolation direction**. At the initial weights ($\alpha \approx 0$), the accuracy is very low, close to random prediction for a 10-class problem. As α increases, the accuracy improves rapidly, especially between $\alpha \approx 0.2$ and 0.6, which corresponds to the region where the loss drops sharply. After $\alpha \approx 0.7$, the accuracy reaches around 97–99% and remains stable even when α becomes slightly larger than 1. This indicates that the trained model lies in a **stable region of the parameter space**, where small perturbations to the weights do not significantly affect performance. Overall, the results suggest that the optimization path is smooth and that the final solution lies in a broad, well-behaved minimum.

Conceptual connection to optimization theory: This work illustrates one of the key concepts from optimization theory discussed in the lecture slide 12. It shows that although neural networks optimize a **high-dimensional, non-convex loss function**, the loss surface can still appear smooth along meaningful directions. In the code, the interpolation formula creates a line between the **initial random weights** and the **trained weights**.

$$\theta(\alpha)=(1-\alpha)\theta_{init}+\alpha\theta_{final}$$

By computing the loss and accuracy along this line, we visualize a **1-dimensional slice of the loss landscape**. The results show that the loss decreases smoothly, and the accuracy increases steadily as α moves from 0 to 1, indicating that the optimizer followed a stable path toward a better solution.

My observation from this is that, even though neural network optimization is theoretically complex and non-convex, the loss surface often behaves smoothly along important directions. The relatively flat region near the final weights suggests the model converged to a **wide minimum**, which is also desirable because wide minima are more stable and often generalize better to unseen data.

Reflection on Challenges: In this work, I have chosen a small number of **epochs (around 10–12)** to keep the training efficient in a **low-GPU Google Colab environment**, that could still allow the model to learn meaningful patterns. I also used **Early stopping** so that the training stops once the validation loss stops improving, preventing unnecessary computation. After some trial and error, a **batch size of 128** was selected because it provides a good balance between stable gradient updates and computational efficiency for MNIST.

One final challenge was obtaining a good accuracy. We can see that, the accuracy appears **very low near $\alpha = 0$** in the interpolation plots because those points correspond to the **initial random weights**, where the model has not learned any features yet and therefore performs close to random guessing for a 10-class classification problem. As the parameters move toward the trained weights ($\alpha \rightarrow 1$), the model learns useful patterns and the accuracy increases significantly.